C++

Build abstractions without giving up control

THE MENTAL MODEL

Design reusable machines, but know who owns each component.

TYPES → OBJECTS → COMPILE → EXECUTE

READ THE CODE

```
#include <iostream>
#include <vector>
int main() {
  std::vector<int> xs{10,20,30};
  for (int x : xs) std::cout << x << " ";
}
```

FOUR CLUES TO READ IT

- #include brings library declarations.
- std::vector<int> is a resizable list of integers.
- for (int x : xs) traverses the list.
- std::cout sends text to standard output.

WHAT MAKES IT DIFFERENT

- Templates and classes enable reusable, strongly typed designs.
- Resource Acquisition Is Initialization (RAII) ties cleanup to object lifetime.

WHERE IT FITS

Game engines, graphics, performance-critical tools and libraries.

WATCH OUT

Powerful abstractions do not automatically prevent memory errors.

TRY ALONE • 60 SEC

Change std::vector<int> to std::vector<double>; what changes?